Graded Project – DevAnswers (React 1)

Introduction

In this project, you will build the frontend of **DevAnswers** - a Q&A web application for developers. Some scaffolding code (layouts, pre-built components, and static question data) is already provided. Your task is to implement the core components that bring the full UI to life.

This project will help you practice:

- Creating React components with props, state, and conditional rendering
- Building controlled forms with validation and user feedback
- Using `useEffect` and `useState` hooks for data loading and state management
- Rendering dynamic lists and handling loading, empty, and error states
- Styling components with companion CSS files
- Writing unit tests for components and pages using React Testing Library
- Using Github Copilot to help with all the tasks above

Setup and Execution

1. Download the project zip file and extract it.
2. Navigate to the `devanswers-frontend` folder:

   ```
   cd devanswers-frontend
   ```

3. Install dependencies:

   ```
   npm install
   ```

4. Start the development server:

   ```
   npm run dev
   ```

5. Open the app in your browser at `http://localhost:5173`. You should see the Header, a sidebar with navigation links, and a blank main content area.
6. To switch between pages during development, uncomment the desired page in `src/App.jsx`:

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

```
<Home />;
{  /* <QuestionDetail id={"q1"} /> */  }
{  /* <Login /> */  }
{  /* <Register /> */  }
```

Project Structure - Files to Implement

You will have to implement the following in the project:

```
starter_code/
├── src/
│   ├── components/
│   │   ├── Answer/
│   │   │   ├── AnswerForm.css
│   │   │   ├── AnswerForm.jsx
│   │   │   ├── AnswerList.css
│   │   │   └── AnswerList.jsx
│   │   ├── Footer/
│   │   │   ├── Footer.css
│   │   │   └── Footer.jsx
│   │   ├── Question/
│   │   │   └── QuestionList.jsx
│   └── pages/
│       └── Question/
│           ├── Home.jsx
│           └── QuestionDetail.jsx
└── tests/
    └── unit/
        ├── components/
        │   └── QuestionList.test.jsx
        └── pages/
            ├── Home.test.jsx
            └── QuestionDetail.test.jsx
```

Problem Statement

Part 1: Footer (`src/components/Footer/`)

- **Footer.jsx** — Render a `<footer>` element displaying `© [Current Year] DevAnswers. All rights reserved.`
- **Footer.css** — Generate styles for the footer element

Part 2: Answer Components (`src/components/Answer/`)

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

a. **AnswerForm.jsx**: This form allows users to submit answers to questions.

- Build a controlled form with a `'Your Answer'` heading, a `<textarea rows={8}>` (placeholder `'Write your answer here...'`), and a **Post Answer** submit button
- On submit, display an alert 'Answer cannot be empty!' if the textarea is blank; otherwise display an alert 'Answer submitted!' and reset the textarea to an empty string. You do not need to implement actual data submission for this project, just simulate the behavior with alerts.
- **AnswerForm.css**: Generate styles for the form card, textarea, and submit button.

b. **AnswerList.jsx**: This component displays a list of answers for a question. It takes an `answers` prop, which is an array of answer objects (or `undefined` if no answers). Each answer object has the following fields (see `data/questions.js` for sample data):

```
{
   "_id",
   "questionId",
   "answerText",
   "author",
   "upvotes",
   "downvotes",
   "voteCount",
   "comments"
}
```

- Show an answer count heading; show an empty-state message when the array is empty
- Each card: answer text, author name, formatted date, and `VoteButtons` with upvote/downvote alerts
- **AnswerList.css**: Style answer cards with alternating even/odd backgrounds

Part 3: QuestionList (`src/components/Question/QuestionList.jsx`):

This component renders a list of questions on the Home page. It receives `questions` and `loading` as props and conditionally renders content based on their values:

- If `loading` is `true`, show a Bootstrap `Spinner` with a loading message
- If `questions` is empty or falsy, show a "No questions found" empty-state message
- Otherwise map `questions` to `<QuestionCard>` components
- QuestionList.css is already provided, but feel free to modify it if needed

Part 4: Home Page (`src/pages/Question/Home.jsx`)

This page displays the list of questions and an "Ask Question" button.

- Implement a `useEffect` hook to load questions from `data/questions.js` into local state on mount

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

- Create a `loading` boolean state to simulate a loading delay (loading is set to `true` before loading starts, and set to `false` after questions are loaded)
- Display a `'All Questions'` heading, a question count, and an **Ask Question** button that simply prints an alert 'Ask Question clicked' when clicked
- Pass `questions` and `loading` to `QuestionList` component;

Part 5: Question Detail Page (`src/pages/Question/QuestionDetail.jsx`)

This page shows the full details of a single question, including its content, answers, and an answer submission form. It receives an `id` prop corresponding to the question's `_id` field.

- Implement a `useEffect` to find the matching question from the set of questions in `data/questions.js` using the `id` prop
- Create a `loading` boolean state to simulate a loading delay (loading is set to `true` before loading starts, and set to `false` after question is loaded)
- Show a loading spinner while loading
- Render the `QuestionContent`, `AnswerList`, and `AnswerForm` components with the appropriate props

Part 6: Tests

- **`tests/unit/components/QuestionList.test.jsx`** — Write five unit tests for `QuestionList` component. Use a mock questions array to test the correct rendering of question title, description, vote counts, etc based on the props passed to `QuestionList`.
- **`tests/unit/pages/Home.test.jsx`** — Write five unit tests for `Home` page component. Test the rendering of the 'Ask Question' button, the loading spinner, the question count, and the question details.
- **`tests/unit/pages/QuestionDetail.test.jsx`** — Write five unit tests for `QuestionDetail` page component. Test the rendering of the question content, answers, answer form, and loading spinner.

Run tests from the project root with the following command:

```
npm test
```

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.